

Desarrollo de Productos Digitales

Nota Técnica — Unidad 4

Licenciatura en Tecnologías Digitales · Taller de Emprendedurismo en Innovación Digital

Profesores: Cottini, Juan Manuel · Foricher, Tomás · Moneda, Pedro · Rshaid, Bernardo

Ciclo lectivo: [Ciclo lectivo]

Cómo leer esta guía

Esta Nota Técnica es el material de estudio principal de la Unidad 4 del Taller. Está pensada para leerse de corrido una primera vez —para entender la lógica del conjunto— y para volver a consultarse por partes mientras trabajás en tu proyecto.

A lo largo del texto vas a encontrar tres tipos de recuadro:

-  **Conexión con la bibliografía.** Te indica qué leer, en qué orden y con qué pregunta en la cabeza. Cuando aparece una frase del tipo “*esta sección debe leerse en tándem con...*”, no es un adorno: es una instrucción de lectura.
-  **Mito vs. realidad.** Desarma las confusiones más frecuentes y más caras sobre cada tema.
-  **Llevalo a la práctica.** Consignas concretas para aplicar el contenido a tu proyecto grupal.

Cada parte cierra con **preguntas disparadoras**. No tienen respuesta única ni se corrigen: están para que discutas con tu equipo antes de la clase siguiente.

Objetivos de aprendizaje

Al finalizar esta unidad, deberías ser capaz de:

1. **Distinguir** los supuestos que sostienen al modelo en cascada y a los enfoques ágiles, y **justificar** cuál conviene a un contexto dado a partir del nivel de incertidumbre y del costo del error.
2. **Explicar** la arquitectura de Scrum —responsabilidades, eventos y artefactos— y la de Kanban —principios, prácticas y métricas de flujo—, señalando qué problema resuelve cada elemento.
3. **Formular** hipótesis de negocio falsables y **diseñar** un Producto Mínimo Viable que las ponga a prueba con el menor esfuerzo posible.
4. **Seleccionar** el tipo de MVP adecuado al riesgo que necesitás despejar y **definir** las métricas que van a informar la decisión de iterar, pivotar o perseverar.
5. **Aplicar** las cinco etapas del Design Thinking a la fase de descubrimiento de tu proyecto.
6. **Integrar** los tres marcos —Design Thinking, Lean Startup y Agile— en un único proceso trazable, desde el insight de una entrevista hasta un ítem entregado.

Dónde estás parado en el recorrido del Taller

Esta unidad no empieza de cero. En la **Unidad 1** construiste un mapa de empatía y una propuesta de valor canvas: definiste *para quién* y *qué problema*. En la **Unidad 3** segmentamos, elegiste un target, estimaste el tamaño de tu mercado y armaste el marketing mix: definiste *cuánta gente* y *cómo llegar*.

La Unidad 4 responde la pregunta que queda: **¿cómo se construye efectivamente eso que prometiste, sin fundir el proyecto en el intento?** Y produce el insumo que vas a necesitar en la **Unidad 5**: los supuestos con los que vas a proyectar tu flujo de fondos y calcular el VAN que se te exige en el Pitch Final. No es una unidad “técnica” aislada. Es el puente entre la idea y el número.

Introducción · Tenés la idea validada. ¿Y ahora cómo se construye?

El salto que casi nadie planifica

Hay un momento incómodo en todo emprendimiento digital. Es el momento en que el equipo ya tiene el canvas lleno de post-its, el mapa de empatía impreso en la pared y un elevator pitch que suena convincente... y alguien pregunta: “*bueno, ¿y ahora quién lo hace?*”.

Ese salto —del papel al producto— es donde mueren la mayoría de los proyectos. No mueren por falta de ideas. Mueren porque el equipo dedica ocho meses y todo su capital a construir algo completo, elegante y bien programado que, cuando finalmente sale, nadie quiere usar.

El dato es tan constante que se volvió casi un lugar común del ecosistema: en el análisis de 110 autopsias de startups realizado por CB Insights (2021), el **35 % de los fundadores** identificó como causa principal del fracaso la ausencia de necesidad de mercado (*no market need*), apenas por debajo de quedarse sin caja (38 %) y muy por encima de ser superado por la competencia (20 %). Dicho de otro modo: la principal causa de muerte de un producto digital no es construirlo mal. Es **construir bien lo que nadie pidió**.

Esta unidad trata, entera, sobre cómo evitar eso.

Las cuatro fases de la creación de un producto digital

Antes de entrar en cada tema conviene tener el mapa completo. Un **producto digital** es una solución intangible que se entrega y que resuelve un problema o satisface una necesidad a través de la tecnología. Su creación atraviesa cuatro fases que vamos a usar como espina dorsal de toda la Nota:

Fase	Pregunta que responde	Herramientas centrales	Resultado esperado
Discovery	¿Cuál es el problema real y de quién es?	Investigación de mercado, entrevistas en profundidad, análisis de datos de comportamiento	Un problema claramente definido que vale la pena resolver

Fase	Pregunta que responde	Herramientas centrales	Resultado esperado
Definition	¿Qué le vamos a ofrecer y por qué lo elegiría?	Mapa de empatía, propuesta de valor canvas, definición del alcance	Una propuesta de valor explícita y acotada
Development	¿Cuál es la versión más chica que prueba nuestra apuesta?	Definición del MVP, backlog, Scrum o Kanban	Un producto mínimo construido y en manos de usuarios
Delivery	¿Qué aprendimos y qué hacemos con eso?	Métricas accionables, feedback cualitativo, análisis de cohortes	La decisión de iterar, escalar o pivotar

Observá la simetría con el recorrido del Taller: *Discovery* y *Definition* son, esencialmente, lo que trabajaste en las Unidades 1 y 3. *Development* y *Delivery* son el contenido de esta unidad.

La tesis de esta guía

Los cuatro temas del programa —cascada vs. agile, Kanban y Scrum, MVP, Design Thinking— suelen enseñarse como cuatro cajas separadas. Es una forma cómoda de organizar un temario y una forma pésima de entender el problema.

La tesis de esta Nota es que los cuatro son **un mismo sistema de reducción de incertidumbre**, y que cada uno ataca un tipo distinto de duda:

- El **Design Thinking** reduce la incertidumbre sobre el **problema**: ¿entendimos bien a quién le duele y qué le duele? (deseabilidad)
- El **MVP** y el ciclo Lean reducen la incertidumbre sobre el **modelo de negocio**: ¿alguien lo usaría, lo pagaría, volvería? (viabilidad)
- **Agile, Scrum y Kanban** reducen la incertidumbre sobre la **ejecución**: ¿podemos construirlo, en qué orden, a qué ritmo y con qué calidad? (factibilidad)

Cuando un equipo aplica uno solo de los tres, el resultado es predecible. Design Thinking sin MVP produce equipos enamorados de insights que nunca se ponen a prueba. MVP sin Design Thinking produce experimentos rigurosos sobre el problema equivocado. Agile sin ninguno de los dos produce equipos que entregan, cada dos semanas, funcionalidades que nadie usa — con excelente velocidad y pésima dirección.



Conexión con la bibliografía

Esta guía debe leerse en tándem con la Primera Parte de *El método Lean Startup* (Ries, 2011), especialmente los capítulos 1 a 3, donde se introduce la noción de **aprendizaje validado**. Guardá esta pregunta mientras lees: ¿qué diferencia hay entre “avanzar” y “aprender”? Toda la unidad gira alrededor de esa distinción.

Y guardá también una segunda conexión, hacia adelante: cada hipótesis que aprendas a formular en la Parte III es, tres unidades después, una fila de tu flujo de fondos. Cuando en la Nota Técnica *Construcción del Flujo de Fondos* se te pide estimar ventas, ticket promedio o penetración, lo que en realidad se te está pidiendo es que justifiques supuestos. El MVP es la máquina de fabricar esas justificaciones.

Parte I · Cómo se construye software: Cascada vs. Agile

1.1 El problema de fondo: incertidumbre y costo del cambio

Toda discusión sobre metodologías de desarrollo es, en el fondo, una discusión sobre **cuándo tomar decisiones**.

Construir un puente y construir una aplicación son actividades distintas en un aspecto decisivo: en el puente, cambiar de opinión a mitad de obra es carísimo; en el software, es apenas caro. Pero hay algo más importante todavía: en el puente, el cliente sabe desde el día cero qué quiere (que una orilla se conecte con la otra, soportando cierto peso). En el software —y muy especialmente en un producto digital nuevo, que todavía no existe en el mercado— **nadie sabe exactamente qué quiere hasta que ve algo funcionando**.

Esa asimetría genera dos estrategias posibles, y las dos son razonables:

- **Decidir todo temprano** y ejecutar con disciplina. Si los requisitos son estables y conocidos, planificar en detalle al inicio maximiza la eficiencia: se evita retrabajo, se puede presupuestar, se puede contratar por resultado.
- **Decidir lo mínimo indispensable y postergar el resto** hasta tener información. Si los requisitos son inciertos, comprometerse temprano equivale a apostar con los ojos vendados: cada decisión tomada sin evidencia es una deuda que se paga más adelante.

Cascada es la primera estrategia llevada a su forma más pura. Agile es la segunda.

1.2 El modelo en cascada: origen y estructura

El modelo en cascada organiza el desarrollo como una **secuencia lineal de fases**, donde cada una debe completarse y aprobarse antes de que comience la siguiente: relevamiento de requisitos → análisis → diseño → implementación → pruebas → despliegue y mantenimiento. La imagen del agua cayendo de un escalón al siguiente —de donde viene el nombre— captura bien la idea: el flujo va en una sola dirección.

Su racionalidad es la de la ingeniería industrial clásica: si documentás exhaustivamente qué hay que hacer antes de empezar a hacerlo, podés estimar plazos y costos, dividir el trabajo entre especialistas y verificar al final que lo entregado coincide con lo pedido.

Vale la pena una aclaración histórica, porque es un caso raro de malentendido académico con consecuencias millonarias. El artículo que suele citarse como acta de nacimiento del modelo, *Managing the Development of Large Software Systems* de Winston Royce (1970), **describe el esquema secuencial para inmediatamente advertir que es riesgoso e invita al fracaso**. Royce proponía, ya en 1970, iterar al menos dos veces, construir un prototipo y involucrar al cliente durante el proceso. La industria se quedó con el diagrama de la primera página y descartó las siete páginas siguientes de advertencias. Una lección temprana sobre los peligros de leer solo el gráfico.

1.3 Ventajas y desventajas del modelo en cascada

Ventajas. La **predictibilidad** es la principal: un plan detallado permite comprometer fecha y precio, algo que un cliente externo o un organismo público suele exigir. La **trazabilidad** es la segunda: cada requisito documentado puede rastrearse hasta su implementación y su prueba, lo que resulta indispensable en dominios regulados (dispositivos médicos, aviónica, sistemas bancarios de core). La **claridad de roles** facilita coordinar equipos grandes o distribuidos, y la documentación exhaustiva reduce la dependencia de personas concretas.

Desventajas. El **feedback llega tarde**, y esa es la crítica central: el usuario ve el producto recién al final, cuando corregir un error de concepción cuesta un orden de magnitud más que al principio. La **entrega es big-bang**: no hay valor entregado hasta el último día, de modo que un proyecto cancelado al 80 % de avance produce cero retorno. La **rigidez ante el cambio** convierte cualquier aprendizaje en un conflicto contractual. Y hay un problema más sutil: el modelo asume que el cliente puede **especificar** lo que necesita, cuando en productos nuevos el cliente recién descubre lo que necesita al usarlos.

1.4 La reacción ágil: el Manifiesto de 2001

En febrero de 2001, diecisiete profesionales del software se reunieron en Snowbird, Utah, y redactaron el **Manifiesto por el Desarrollo Ágil de Software** (Beck et al., 2001). El texto es asombrosamente breve —cuatro valores y doce principios— y su formulación es deliberadamente comparativa:

Estamos descubriendo formas mejores de desarrollar software tanto por nuestra propia experiencia como ayudando a terceros. A través de este trabajo hemos aprendido a valorar:

*Individuos e interacciones **sobre** procesos y herramientas Software funcionando **sobre** documentación extensiva Colaboración con el cliente **sobre** negociación contractual Respuesta ante el cambio **sobre** seguir un plan*

Esto es, aunque valoramos los elementos de la derecha, valoramos más los de la izquierda.

Esa última línea es la más citada y la menos leída. El Manifiesto **no dice** que la documentación, los contratos o los planes no sirvan. Dice que, ante un conflicto entre ambas columnas, hay que privilegiar la izquierda.

De los doce principios, cuatro son especialmente relevantes para un emprendedor: la satisfacción del cliente mediante **entrega temprana y continua** de software con valor; la aceptación de que **los requisitos cambien, incluso en etapas tardías**, como ventaja competitiva; la entrega de software funcionando **con frecuencia**, preferentemente en el lapso más corto posible; y el criterio de que **el software funcionando es la medida principal de progreso** —no el porcentaje de avance del plan, no la cantidad de documentos aprobados.

1.5 Ventajas y desventajas de los enfoques ágiles

Ventajas. La **entrega temprana de valor** permite que el producto empiece a generar retorno —y aprendizaje— antes de estar terminado. La **adaptación** convierte el cambio de requisitos en un evento normal en lugar de una crisis. El **riesgo se distribuye**: en lugar de una única apuesta grande, se hacen muchas apuestas chicas y corregibles. Y la **motivación del equipo** tiende a ser mayor, porque los equipos autoorganizados que ven su trabajo en manos de usuarios reales se involucran más que los que completan tareas asignadas.

Desventajas. El **alcance y el presupuesto quedan abiertos**, lo que complica cualquier contrato a precio y fecha cerrados. La **dependencia de la disponibilidad del cliente** es alta: sin alguien que priorice y valide de manera continua, el proceso se degrada. La **deuda técnica** se acumula con facilidad si la presión por entregar rápido no se equilibra con estándares de calidad explícitos. La **documentación insuficiente** genera problemas de continuidad cuando rota el equipo. Y existe el riesgo más extendido de todos: el **“agile de cartón”**, equipos que adoptan la ceremonia —hacen dailies, llaman “sprints” a las quincenas— sin adoptar ninguno de los valores, y terminan con cascada disfrazada de ágil, con todos sus costos y ninguno de sus beneficios.

1.6 Cuadro comparativo: Cascada vs. Agile

Dimensión	Cascada (Waterfall)	Agile
Enfoque del alcance	Fijo; se define completo al inicio	Variable; se refina de forma continua
Variable de ajuste	Tiempo y costo (el alcance es intocable)	Alcance (el tiempo y el equipo son fijos)
Momento del feedback	Al final, en pruebas de aceptación	Continuo, al cierre de cada iteración
Gestión del cambio	Excepción a controlar; genera adendas	Expectativa normal; genera aprendizaje
Rol del cliente	Interviene al inicio y al final	Participa de forma permanente
Documentación	Exhaustiva y previa	Suficiente y just-in-time
Entrega	Única, al cierre (big-bang)	Incremental y frecuente
Medida de progreso	Porcentaje del plan ejecutado	Software funcionando en producción
Estructura del equipo	Especializada y jerárquica	Multifuncional y autoorganizada
Modelo contractual	Precio y alcance cerrados	Tiempo y materiales, o alcance variable
Perfil de riesgo	Concentrado al final	Distribuido a lo largo del proyecto
Contexto ideal	Requisitos estables, alto costo del error, exigencias regulatorias	Requisitos inciertos, mercado cambiante, necesidad de aprender rápido

1.7 Las similitudes que casi nunca se mencionan

La literatura de divulgación suele presentar a los dos enfoques como opuestos irreconciliables. Es una simplificación pedagógicamente útil y conceptualmente falsa. Comparten más de lo que se admite:

- **Ambos planifican.** Agile no elimina la planificación: la distribuye. Hay planificación de release, de sprint y diaria. Lo que cambia es el horizonte y el nivel de detalle, no la existencia del plan.
- **Ambos necesitan requisitos.** Una historia de usuario es un requisito; lo que cambia es su granularidad, su formato y el momento en que se define.
- **Ambos testean.** La diferencia está en si el testing es una fase al final o una actividad permanente.
- **Ambos tienen gobierno y roles.** Scrum tiene tres responsabilidades explícitas y cinco eventos obligatorios. Llamarlo “trabajar sin estructura” es no haberlo leído.
- **Ambos fracasan por las mismas causas humanas.** Objetivos poco claros, falta de involucramiento del sponsor, equipos sobrecargados, ausencia de comunicación. Ninguna metodología corrige un problema organizacional.

1.8 El falso dilema y cómo elegir

En la práctica profesional casi nadie aplica ninguno de los dos modelos en estado puro. Lo habitual son **híbridos**: un marco general por fases con contrato y presupuesto anual, y ejecución iterativa dentro de cada fase.

Un criterio de decisión razonable combina tres variables:

1. **Nivel de incertidumbre.** ¿Sabemos qué hay que construir, o lo estamos descubriendo? A mayor incertidumbre, más conviene iterar.
2. **Costo del error.** ¿Cuánto cuesta equivocarse y corregir? Si una falla implica riesgo de vida o sanción regulatoria, la validación previa exhaustiva se justifica.
3. **Velocidad de cambio del entorno.** Si el mercado, la tecnología o la regulación se mueven rápido, un plan a dieciocho meses nace obsoleto.

Para tu proyecto del Taller la respuesta es casi siempre la misma, y conviene que entiendas por qué. Estás desarrollando un producto que no existe, para un mercado que todavía no validaste, con un equipo de cinco personas sin dedicación full-time y un horizonte de un cuatrimestre. Las tres variables apuntan en la misma dirección: máxima incertidumbre, bajo costo del error, entorno móvil. Ahora bien, cuando en el Pitch Final digas “vamos a trabajar con metodologías ágiles”, el jurado va a esperar que puedas justificar **por qué** —con estos tres criterios— y no simplemente que lo enuncies como una contraseña.



Mito vs. realidad

Mito 1: “Ágil significa trabajar sin documentación.” Falso. El Manifiesto dice *documentación suficiente*, no *nula*. Un equipo ágil documenta decisiones de arquitectura, criterios de aceptación y APIs; lo que no hace es redactar 200 páginas de especificación que quedarán obsoletas antes de escribirse la primera línea de código.

Mito 2: “Ágil significa sin plan y sin fechas.” Falso. Significa plan de horizonte corto, revisado con frecuencia. Un equipo ágil sabe qué va a entregar en dos semanas con mucha más precisión que un equipo en cascada al mes ocho de dieciocho.

Mito 3: “Cascada es obsoleta.” Falso. Sigue siendo el estándar en construcción, aviónica, dispositivos médicos y grandes migraciones de sistemas con alcance conocido. La pregunta correcta no es cuál es mejor, sino cuál corresponde a este problema.

Mito 4: “Ágil es más rápido.” Engañoso. No entrega el producto completo antes: entrega **algo utilizable** —y **aprendizaje**— mucho antes.

Conexión con la bibliografía

Leé dos textos breves, ambos de acceso libre, y compará. Primero, el **Manifiesto Ágil** (Beck et al., 2001): son dos páginas, está traducido al español y lleva veinticinco años sin modificarse. Segundo, el artículo de **Royce (1970)** —o, si te resulta arduo, al menos su primera sección. La consigna es una sola pregunta: *¿en qué se parece la propuesta final de Royce a lo que hoy llamamos ágil?* Vas a descubrir que el supuesto padre de la cascada proponía iterar, prototipar e involucrar al cliente. Como se profundiza en la lectura de Royce, esto significa que la discusión metodológica de los últimos cincuenta años ha sido, en buena medida, una discusión con un espantapájaros.

Preguntas disparadoras

1. Pensá en un proyecto donde cascada sea claramente superior a agile. ¿Qué características lo hacen así?
2. Si tu equipo del Taller tuviera que firmar hoy un contrato a precio cerrado por el desarrollo completo de su producto, ¿qué riesgo estarían asumiendo? ¿Y quién lo asume: ustedes o el cliente?
3. El Manifiesto dice que el software funcionando es la medida principal de progreso. ¿Cuál sería el equivalente de “software funcionando” para medir el avance de tu proyecto en el Taller?
4. ¿Qué significa, concretamente, “documentación suficiente” en un equipo de cinco personas que se ven dos veces por semana?
5. Identificá un caso —de tu experiencia laboral o académica— de “agile de cartón”. ¿Qué ceremonia se copió y qué valor se ignoró?

Parte II · Metodologías ágiles: Kanban y Scrum

2.1 Marco, método y práctica: tres cosas que no son lo mismo

Antes de entrar en detalle conviene ordenar el vocabulario, porque el uso corriente lo mezcla todo.

Agile no es una metodología. Es un conjunto de valores y principios: define un *para qué* y un *cómo pensar*, pero no dice qué hacer el lunes a la mañana. Es, en rigor, una filosofía de trabajo.

Scrum y Kanban son marcos de trabajo (frameworks). Un marco es deliberadamente incompleto: define un esqueleto —eventos, roles, artefactos, reglas— y deja que cada equipo lo complete con sus propias prácticas. Por eso Scrum no te dice cómo estimar, ni qué herramienta usar, ni cómo escribir una historia de usuario. Esa incompletitud es intencional: lo que funciona en un equipo de siete desarrolladores de una fintech no funciona en un equipo de tres diseñadores de una agencia.

Las prácticas concretas —historias de usuario, planning poker, integración continua, pair programming, tableros físicos— son el relleno. Se toman de donde sirvan.

La consecuencia práctica: cuando alguien dice “nosotros usamos Scrum”, la afirmación por sí sola informa poco. La pregunta útil es qué hacen en cada evento y qué artefactos mantienen realmente vivos.

2.2 Scrum: el fundamento empírico

Scrum se apoya en el **empirismo**: el conocimiento proviene de la experiencia, y las decisiones se toman sobre la base de lo observado, no de lo supuesto. De ahí se derivan sus tres pilares (Schwaber & Sutherland, 2020):

- **Transparencia.** El proceso y el trabajo deben ser visibles para quienes ejecutan y para quienes reciben. Sin transparencia, la inspección engaña.
- **Inspección.** Los artefactos y el avance hacia los objetivos se revisan con frecuencia, para detectar desviaciones temprano.
- **Adaptación.** Si la inspección revela una desviación, el proceso o el producto se ajustan **inmediatamente**, no en el próximo trimestre.

Scrum añade cinco valores que sostienen esos pilares: **compromiso, foco, apertura, respeto y coraje**. Parecen decorativos y no lo son: el coraje, por ejemplo, es lo que permite que un desarrollador diga en la Daily que no va a llegar, en lugar de esconderlo hasta el último día.

2.3 Las tres responsabilidades

La Guía Scrum 2020 dejó de hablar de “roles” y pasó a hablar de **responsabilidades** (*accountabilities*), justamente para evitar la lectura jerárquica. Un Scrum Team es un grupo de diez personas o menos, sin subequipos ni jerarquías internas, con tres responsabilidades:

Product Owner. Responsable de **maximizar el valor** del producto. Es quien define el Product Goal, crea y ordena el Product Backlog y asegura que sea transparente y entendido. Es una persona, no un comité — aunque pueda representar a muchos interesados. Su decisión sobre prioridades no se sobrescribe: si alguien quiere cambiar el orden del backlog, tiene que convencerlo a él.

Scrum Master. Responsable de que Scrum se entienda y se aplique, y de la **efectividad del equipo**. No es un jefe de proyecto ni asigna tareas: es un líder al servicio del equipo. Su trabajo real consiste en remover impedimentos, facilitar los eventos y proteger al equipo de interrupciones externas.

Developers. Todos los que construyen el incremento, cualquiera sea su especialidad: programadores, diseñadores, testers, analistas. Son responsables de crear el plan del Sprint, adherir a la Definition of Done y ajustar su plan diariamente.

Traducción a tu equipo del Taller. En un grupo de cinco, estas responsabilidades no requieren cinco personas distintas. Alguien tiene que ser el dueño de la visión del producto y decidir qué se construye primero (Product Owner). Alguien tiene que velar por que el equipo funcione, que las reuniones sirvan y que los bloqueos se destraben (Scrum Master). El resto —y también ellos— construyen. Lo que no funciona es que las decisiones de prioridad se tomen por votación permanente: es la forma más rápida de que un proyecto de un cuatrimestre no llegue a ningún lado.

2.4 Artefactos y compromisos

Scrum define tres artefactos, cada uno con un **compromiso** asociado que le da foco y permite medir el avance:

Artefacto	Qué es	Compromiso asociado
Product Backlog	Lista ordenada y emergente de todo lo que podría necesitar el producto. Única fuente de trabajo.	Product Goal: el objetivo de largo plazo hacia el que avanza el equipo
Sprint Backlog	El subconjunto seleccionado para este Sprint, más el plan para entregarlo.	Sprint Goal: el único objetivo del Sprint, que da coherencia al conjunto
Increment	El escalón concreto y utilizable hacia el Product Goal, sumado a todos los anteriores.	Definition of Done: el estándar de calidad que define cuándo algo está terminado

Dos comentarios sobre lo que más se descuida.

El **Sprint Goal** es lo que diferencia un Sprint de una lista de tareas. Si al terminar el Sprint el equipo no puede decir en una frase qué logró, no había Sprint Goal: había un rejunte.

La **Definition of Done** es un acuerdo explícito y escrito. “Terminado” no puede significar cosas distintas para cada integrante. Una DoD mínima para un proyecto del Taller podría ser: *el código está integrado en la rama principal, fue revisado por otro integrante, pasa las pruebas, está desplegado en el ambiente de prueba y tiene una nota de uso de un párrafo*. Sin DoD escrita, el “90 % terminado” se extiende indefinidamente.

Los ítems del backlog suelen escribirse como **historias de usuario**: *Como [tipo de usuario], quiero [acción] para [beneficio]*. El formato importa menos que lo que fuerza a explicitar — quién, qué y, sobre todo, **para qué**. Un buen ítem cumple los criterios **INVEST**: independiente, negociable, valioso, estimable, pequeño y testeable.

2.5 Los cinco eventos

El Sprint es el contenedor de todos los demás: un período de **un mes o menos**, de duración fija, que comienza inmediatamente después del anterior. Durante el Sprint no se hacen cambios que pongan en riesgo el Sprint Goal, y el alcance puede renegociarse con el Product Owner a medida que se aprende.

Evento	Duración máxima (Sprint de 1 mes)	Propósito	Error típico
Sprint Planning	8 horas	Definir el Sprint Goal, seleccionar los ítems y planificar cómo se harán	Planificar tareas sin acordar el objetivo

Evento	Duración máxima (Sprint de 1 mes)	Propósito	Error típico
Daily Scrum	15 minutos, diario	Inspeccionar el avance hacia el Sprint Goal y ajustar el plan del día	Convertirla en un reporte de estado al Scrum Master
Sprint Review	4 horas	Mostrar el incremento a los interesados y adaptar el backlog según el feedback	Demo de PowerPoint en lugar de producto funcionando
Sprint Retrospective	3 horas	Inspeccionar cómo trabajó el equipo y comprometer mejoras	Catarsis sin acciones concretas ni responsables
Sprint	1 mes	Contener a todos los demás y producir un incremento	Extenderlo “una semanita más” para terminar

2.6 Un Sprint narrado

La teoría se entiende mejor en movimiento. Supongamos un equipo de cinco que desarrolla un marketplace de servicios para barrios cerrados, con Sprints de dos semanas.

Lunes, Planning (2 horas). El Product Owner presenta el Product Goal —“*que un vecino pueda contratar y pagar un servicio sin salir de la app*”— y propone los ítems de arriba del backlog. El equipo discute, pregunta, descarta uno por demasiado grande y acuerda el **Sprint Goal**: “*un vecino puede buscar un proveedor, ver su disponibilidad y reservar un turno*”. El pago queda fuera: es el Sprint siguiente. Se seleccionan seis ítems y se desagregan en tareas.

Todos los días, Daily (15 minutos, de pie). No se recorre a las personas una por una. Se mira el tablero y se pregunta por el objetivo: *¿qué nos falta para el Sprint Goal? ¿qué nos está frenando?* El miércoles aparece un impedimento —la API de calendario que iban a usar exige una cuenta paga— y el Scrum Master se lleva la tarea de resolverlo fuera de la reunión.

Jueves de la segunda semana. Queda claro que uno de los seis ítems no va a entrar. El equipo habla con el Product Owner: como no compromete el Sprint Goal, se devuelve al backlog. Esto **no es un fracaso**: es exactamente el mecanismo funcionando.

Viernes, Review (1 hora). Se muestra el flujo completo funcionando ante dos vecinos invitados. Uno de ellos hace una observación que nadie había previsto: quiere ver la calificación del proveedor *antes* de elegir el horario. Se anota como ítem nuevo.

Viernes, Retrospective (45 minutos). El equipo detecta que perdió dos días por no haber validado la API antes de comprometerla. Acción concreta para el próximo Sprint: *toda dependencia externa se prueba con un spike de 2 horas antes de incluirse en el Sprint Backlog*. Con responsable y fecha.

Observá lo que acaba de pasar: el equipo entregó valor, aprendió algo del usuario, aprendió algo de sí mismo y ajustó su proceso. En dos semanas.

2.7 Kanban: origen y principios

Kanban —del japonés *kan* (visual) *ban* (tarjeta)— nació en el Sistema de Producción Toyota como mecanismo de señalización para producir solo lo que la etapa siguiente consumía. Su traslado al trabajo del conocimiento, sistematizado por David Anderson, conserva la idea central: **el trabajo se tira (*pull*), no se empuja (*push*)**.

A diferencia de Scrum, Kanban **no propone romper con lo que ya existe**. Sus cuatro principios fundacionales son explícitamente conservadores (Anderson & Carmichael, 2016):

1. Empezá con lo que hacés ahora.
2. Acordá perseguir la mejora mediante cambios evolutivos e incrementales.
3. Respetá los procesos, roles y responsabilidades actuales.
4. Fomentá el liderazgo en todos los niveles.

Esto lo vuelve especialmente adecuado para organizaciones que no pueden —o no quieren— reorganizarse, y para equipos cuyo trabajo no encaja en iteraciones: soporte, mantenimiento, operaciones, marketing.

2.8 Las seis prácticas

1. **Visualizar el flujo de trabajo.** Un tablero con una columna por etapa real del proceso. La clave está en *real*: si entre “En desarrollo” y “Terminado” hay en los hechos una espera de revisión de tres días, esa espera merece su propia columna. Lo que no se ve, no se gestiona.
2. **Limitar el trabajo en curso (WIP).** Cada columna tiene un número máximo de tarjetas. Es la práctica más contraintuitiva y la más potente: **limitar cuánto se empieza es lo que hace que las cosas se terminen**.
3. **Gestionar el flujo.** El foco no está en las personas ocupadas sino en el movimiento del trabajo. Se busca reducir esperas y bloqueos, no aumentar la utilización.
4. **Hacer explícitas las políticas.** Qué significa que una tarjeta pase de una columna a la siguiente, quién puede moverla, cómo se prioriza una urgencia. Escrito y a la vista.
5. **Implementar ciclos de feedback.** Reuniones regulares de reposición, de revisión de entrega y de mejora del sistema.
6. **Mejorar colaborativamente, evolucionar experimentalmente.** Cambios chicos, medidos, reversibles.

2.9 Las métricas de flujo

Kanban aporta un vocabulario de medición que sirve incluso si no usás Kanban:

- **Lead time:** tiempo total desde que el cliente pide algo hasta que lo recibe. Es la métrica que le importa al cliente.
- **Cycle time:** tiempo desde que el equipo empieza a trabajar en un ítem hasta que lo termina. Es la métrica que controla el equipo.
- **Throughput:** cantidad de ítems terminados por unidad de tiempo.

- **Work in Progress (WIP):** cantidad de ítems empezados y no terminados.
- **Diagrama de flujo acumulado (CFD):** gráfico que muestra la evolución de los ítems en cada estado; las bandas que se ensanchan revelan cuellos de botella.

La relación entre las tres primeras es la **Ley de Little**, y explica por qué limitar el WIP funciona:

$$\text{Tiempo de ciclo} = \text{Trabajo en curso} \div \text{Throughput}$$

Si tu equipo termina 5 ítems por semana (throughput) y mantiene 20 ítems empezados a la vez (WIP), cada ítem tarda 4 semanas en salir. Si reducís el WIP a 5 manteniendo el mismo throughput, el cycle time baja a 1 semana. No trabajaste más rápido: dejaste de trabajar en muchas cosas a la vez. El multitasking tiene un costo que la mayoría de los equipos jóvenes subestima por completo.

2.10 Cuadro comparativo: Scrum vs. Kanban

Dimensión	Scrum	Kanban
Cadencia	Sprints de duración fija	Flujo continuo, sin iteraciones obligatorias
Roles	Tres responsabilidades definidas	No prescribe roles
Compromiso	El equipo se compromete a un Sprint Goal	No hay compromiso por período
Tablero	Se vacía al final de cada Sprint	Es permanente
Cambios	No durante el Sprint (si afectan el Goal)	En cualquier momento, según capacidad
Métrica central	Velocity (puntos por Sprint)	Cycle time y throughput
Limitación del trabajo	Indirecta: por capacidad del Sprint	Directa y explícita: límites de WIP por columna
Cambio organizacional	Alto: exige reestructurar roles y eventos	Bajo: se superpone al proceso existente
Mejor para	Desarrollo de producto nuevo, con objetivos por período	Trabajo con llegada impredecible: soporte, mantenimiento, operaciones
Riesgo típico	Ceremonias vacías, Sprint como microcascada	Tablero decorativo sin límites de WIP

2.11 Scrumban y qué conviene a un equipo chico

En la práctica, muchos equipos combinan: iteraciones y eventos de Scrum con límites de WIP y métricas de flujo de Kanban. Ese híbrido, bautizado **Scrumban**, no es una tercera metodología sino una constatación de que los marcos son cajas de herramientas y no religiones.

Para un equipo del Taller —cinco personas, sin dedicación completa, un cuatrimestre— la recomendación honesta es: **empezá con un tablero Kanban simple y sumá de Scrum solo tres cosas**. El tablero, con límites de WIP, te da visibilidad inmediata y cuesta media hora armarlo. De Scrum, incorporá una reunión de planificación quincenal con un objetivo explícito, una revisión quincenal mostrando algo que funcione ante alguien externo al equipo, y una retrospectiva quincenal con dos acciones concretas. Eso es el 80 % del beneficio con el 20 % de la ceremonia. Implementar Scrum completo en un equipo que se ve dos veces por semana suele producir más teatro que producto.

2.12 Ocho antipatrones frecuentes

Antipatrón	Qué se ve	Qué lo causa
Daily como reporte	Cada uno le cuenta al Scrum Master qué hizo ayer	Se confunde inspección del trabajo con control de personas
Backlog basurero	300 ítems, ninguno priorizado, la mitad obsoletos	Nadie ejerce realmente el rol de Product Owner
Sprint sin Sprint Goal	Una lista de tareas sin hilo conductor	Se planifica por capacidad en lugar de por objetivo
Microcascada	Semana 1 diseño, semana 2 desarrollo, testing “después”	Se adoptó el calendario sin el principio de incremento entregable
WIP infinito	Todo empezado, nada terminado	No hay límites explícitos ni acuerdo para respetarlos
Retro sin acciones	Se habla, se desahoga, nada cambia	No se registran acciones con responsable y fecha
DoD tácita	“Está casi listo” durante tres semanas	No hay estándar escrito de terminación
Velocity como meta	Se inflan las estimaciones para “mejorar”	Se usa una métrica de planificación como métrica de desempeño



Conexión con la bibliografía

Esta sección debe leerse en tándem con dos documentos breves y gratuitos. **La Guía Scrum 2020** (Schwaber & Sutherland) tiene trece páginas, está traducida al español y es la única fuente normativa

de Scrum: todo lo demás es interpretación. Como se profundiza en la Guía, esto significa —entre otras cosas— que el Scrum Master **no es un jefe de proyecto**, que el Sprint no puede acortarse ni extenderse una vez iniciado, y que un incremento que no cumple la Definition of Done simplemente no se presenta en la Review. Leela con un lápiz y marcá cada punto donde tu intuición previa no coincidía con el texto.

Complementariamente, **Essential Kanban Condensed** (Anderson & Carmichael, 2016) resume en unas cincuenta páginas de descarga libre los principios, prácticas y métricas de flujo. Con leer los capítulos sobre límites de WIP y métricas te alcanza para esta unidad.

Llévalo a la práctica: armá tu tablero en 30 minutos

Con tu equipo, y antes de la próxima clase:

1. **Dibujen las columnas reales de su proceso.** No copien “Por hacer / Haciendo / Hecho”. Pregúntense por qué etapas pasa efectivamente una tarea en su equipo. Casi siempre aparece al menos una espera oculta.
2. **Pongan límites de WIP.** Regla práctica para empezar: en las columnas de trabajo activo, el límite es el número de integrantes disponibles. Cinco personas, límite de tres en “Haciendo”. Va a incomodar. Ese es el punto.
3. **Escriban su Definition of Done** en no más de cinco puntos, y péguenla arriba del tablero.
4. **Definan su política de urgencias:** ¿qué pasa cuando aparece algo que no puede esperar? ¿Se rompe el límite de WIP o se saca algo del tablero?
5. **Corran dos semanas y midan una sola cosa:** cuántos ítems terminaron. Ese es su throughput inicial. A partir de ahí, todo lo demás es comparable.

Preguntas disparadoras

1. ¿Por qué limitar el trabajo en curso *acelera* la entrega, si intuitivamente parece que hacer menos cosas a la vez debería producir menos?
2. En tu equipo, ¿quién debería ser el Product Owner y por qué? ¿Qué pasaría si las prioridades se decidieran siempre por consenso?
3. Un equipo hace dailies, sprints y retros, pero el testing se concentra en la última semana y el usuario ve el producto recién al final. ¿Es ágil? Justificá con el Manifiesto en la mano.
4. ¿Qué tres ítems pondrías en tu Definition of Done y cómo verificarías cada uno?
5. Si tu equipo tuviera que elegir **una sola** práctica de esta parte para adoptar esta semana, ¿cuál tendría el mayor impacto y por qué?

Parte III · El Producto Mínimo Viable

3.1 Definición: tres palabras que hay que tomar en serio

MVP son las siglas de *Minimum Viable Product*: producto mínimo viable. Las tres palabras están elegidas con precisión y conviene desarmarlas una por una, porque cada una se malinterpreta de una manera distinta.

Mínimo. Debe contener **solo las características esenciales** para resolver el problema que se quiere validar. No las que al equipo le entusiasman, no las que “total ya que estamos”, no las

que el competidor tiene. Mínimo se define contra una hipótesis: es mínimo respecto de *lo que necesitamos aprender*.

Viable. Debe ser **funcional y proporcionar una experiencia real** al usuario. Esta es la palabra que más se descuida. Un MVP tiene que resolver el problema de verdad para alguien de verdad; si no funciona, lo que vas a medir es la frustración del usuario con tu implementación, no su interés en tu propuesta de valor. Un producto que no cumple su promesa no genera aprendizaje válido: genera ruido.

Producto. Puede ser una aplicación móvil, un sitio web, un servicio, un producto físico o una combinación. No hay ninguna exigencia de que sea software, y algunos de los MVP más efectivos de la historia no tenían una sola línea de código.

Ries (2011) lo define de forma operativa: el MVP es la **versión de un nuevo producto que permite al equipo recolectar la máxima cantidad de aprendizaje validado sobre los clientes con el menor esfuerzo**. Notó que la definición no habla de funcionalidades ni de costos: habla de **aprendizaje por unidad de esfuerzo**. Esa es la métrica que el MVP optimiza.

3.2 Lo que un MVP no es

Buena parte de los errores en esta materia vienen de confundir el MVP con cuatro cosas parecidas:

No es...	Porque...
La versión 1.0 con menos funcionalidades	La v1.0 recortada busca <i>entregar valor con menos</i> ; el MVP busca <i>aprender</i> . Son objetivos distintos y producen diseños distintos
Un prototipo	Un prototipo es una simulación para explorar una solución, generalmente sin usuarios reales ni transacciones reales. El MVP está en el mercado
Una prueba de concepto (PoC)	Una PoC responde <i>¿es técnicamente posible?</i> . El MVP responde <i>¿a alguien le importa?</i> . Son preguntas de riesgo distintas
Un producto mal hecho	“Mínimo” se refiere al alcance, no a la calidad. Dentro de lo que hace, el MVP debe funcionar bien

La confusión más costosa es la primera, y suele delatarse en los pitches con una frase: “*nuestro MVP va a tener las funcionalidades A, B, C, D y E*”. Cinco funcionalidades no es un mínimo: es un producto chico. La pregunta correcta nunca es “*¿qué es lo menos que podemos construir?*” sino “**¿cuál es la apuesta más riesgosa que estamos haciendo, y cuál es la forma más barata de averiguar si es cierta?**”.

3.3 El ciclo Construir–Medir–Aprender

El MVP no es un entregable aislado: es el motor de un ciclo. Ries (2011) lo formula como **Construir → Medir → Aprender**, y señala una particularidad decisiva: aunque las actividades se ejecutan en ese orden, **se planifican en el orden inverso**.

- Empezás por el final: *¿qué necesitamos **aprender**?*
- De ahí derivás: *¿qué necesitamos **medir** para saberlo?*
- Y recién entonces: *¿qué es lo mínimo que necesitamos **construir** para poder medir eso?*

Planificar en el orden natural —construir, después ver qué medimos, después ver qué aprendimos— es la forma habitual de terminar con un producto completo y ninguna conclusión.

El producto de este ciclo es el **aprendizaje validado**: evidencia empírica sobre el comportamiento real de los clientes, obtenida mediante experimentos. No es una opinión, no es una encuesta de intención, no es que a tu tío le pareció buenísimo. Es gente haciendo algo —o no haciéndolo— frente a una oferta concreta.

La distinción importa porque la alternativa es el avance aparente: un equipo puede pasar seis meses cumpliendo su cronograma al 100 % y no haber aprendido absolutamente nada sobre si su negocio existe. Avanzar y aprender no son sinónimos.

3.4 La hipótesis como unidad de trabajo

Si el MVP existe para poner a prueba una apuesta, lo primero es **escribir la apuesta**. Y escribirla de manera que pueda resultar falsa: una hipótesis que no puede fallar no es una hipótesis, es un deseo.

Ries distingue dos tipos de hipótesis fundamentales:

- **Hipótesis de valor:** ¿el producto entrega realmente valor a los usuarios cuando lo usan? ¿Vuelven? ¿Pagarían?
- **Hipótesis de crecimiento:** ¿cómo va a descubrir el producto nuevos usuarios? ¿El mecanismo de crecimiento funciona?

La mayoría de los equipos jóvenes ataca primero la de crecimiento —cómo conseguir usuarios— cuando todavía no validó la de valor. Es construir el embudo antes de saber si hay algo que valga la pena embudar.

Plantilla de hipótesis falsable. Usá este formato, literalmente, para cada apuesta importante de tu proyecto:

Creemos que [segmento específico de usuarios] **tiene el problema de** [problema concreto] **y que al ofrecerle** [solución acotada] **va a** [comportamiento observable y medible]. **Sabremos que estábamos en lo cierto si** [métrica] **alcanza** [umbral] **en** [plazo]. **Si no lo alcanza,** [qué decisión tomamos].

El último renglón es el que casi todos omiten, y es el más importante: **definir el criterio de fracaso antes del experimento**. Si el umbral se define después de ver los datos, el experimento siempre “confirma” la hipótesis. Es el mecanismo psicológico más confiable que existe para no aprender nada.

Un ejemplo, tomado del caso de minimercados autónomos en barrios cerrados que se trabaja en clase. En lugar de “creemos que a la gente le va a gustar”, la hipótesis operativa sería: *creemos que los residentes de barrios privados de más de 200 lotes tienen el problema de que comprar implica salir del barrio, y que al ofrecerles un minimercado autónomo dentro del barrio van a comprar al menos dos veces por semana; sabremos que estábamos en lo cierto si la penetración supera el 25 % de los hogares y el ticket promedio supera los X pesos durante ocho semanas; si no lo alcanza, revisamos si el problema es el surtido o el tamaño mínimo de barrio.*

Fijate todo lo que hay que decidir para escribir esa frase: el segmento, el umbral de tamaño del barrio, la frecuencia esperada, el ticket, el plazo, la penetración mínima, y qué variable revisar si falla. Eso ya es, prácticamente, el modelo de negocio.

3.5 Tipos de MVP

No existe un único formato de MVP. Existe una familia, y elegir el correcto depende de **qué riesgo querés despejar y con qué presupuesto**.

Tipo	En qué consiste	Qué valida	Costo / velocidad	Ejemplo
Concierge	Se brinda el servicio de forma manual y personalizada a los primeros usuarios, sin automatizar nada	Si el problema existe y si la solución resuelve de verdad; permite aprendizaje cualitativo profundo	Bajo costo, alta dedicación por usuario	Servicio de entrega de comidas caseras operado a mano
Mago de Oz	El usuario cree que interactúa con un sistema automatizado, pero detrás hay personas ejecutando	La demanda y la experiencia del flujo completo, sin construir el motor	Bajo costo de tecnología, alto costo operativo	Un “algoritmo” de recomendación que en realidad es una persona eligiendo
Landing page (aterrizaje)	Una página que describe la propuesta y captura interés —mail, preinscripción, clic en “comprar”—	Interés declarado y capacidad de captar atención; sirve para medir conversión y probar mensajes	Muy bajo, horas de trabajo	Página de una app de fitness con lista de espera para acceso anticipado

Tipo	En qué consiste	Qué valida	Costo / velocidad	Ejemplo
Preorden	Se ofrece comprar o reservar el producto antes de que exista	La disposición a pagar, que es la validación más dura que hay	Bajo, pero genera compromiso con el cliente	Dispositivo electrónico que se paga por adelantado
Pieza única	Se desarrolla y lanza una sola funcionalidad del producto completo	Si esa funcionalidad, por sí sola, genera uso recurrente	Medio	Plataforma legal que arranca solo con consultas en línea
Producto falso	Se simula el funcionamiento sin desarrollar la funcionalidad real (por ejemplo, un botón que registra el clic y avisa “próximamente”)	La demanda de una función antes de construirla	Muy bajo	Botón de suscripción a contenido exclusivo que aún no existe

Dos advertencias sobre esta tabla.

La primera: los tipos **se combinan**. Un MVP realista suele ser una landing que captura interesados, a los que se les da un servicio concierge mientras se mide si vuelven.

La segunda, de orden ético: el *producto falso* y el *magos de Oz* implican cierto grado de ocultamiento sobre lo que el usuario está recibiendo. La línea que no conviene cruzar es la del daño: está bien que el usuario no sepa que detrás del sistema hay una persona; no está bien cobrarle por algo que no vas a entregar, ni capturar datos sensibles sin propósito real, ni hacerle perder tiempo significativo sin contrapartida. Un experimento que quema a tus primeros usuarios te deja sin el activo más escaso que tenés al principio: gente dispuesta a probar lo que hacés.

3.6 Los cinco pasos para construir un MVP

1. **Problema.** Comprender el problema que se busca resolver y las necesidades del mercado objetivo. Es el trabajo de la fase de Discovery — y es donde el Design Thinking de la Parte IV entra en escena.
2. **Definir hipótesis.** Establecer explícitamente los supuestos sobre el producto y su mercado, con la plantilla de 3.4. Ordenarlos por riesgo: primero el supuesto que, de ser falso, hace inviable todo lo demás.
3. **Diseño.** Seleccionar las funcionalidades mínimas necesarias **para validar esas hipótesis** y crear una experiencia de usuario básica pero real.

4. **Construir y lanzar.** Desarrollar de forma ágil y ponerlo en manos de usuarios. Lanzar chico y controlado: un grupo acotado permite corregir sin exponerse.
5. **Medir y aprender.** Recopilar datos cuantitativos y feedback cualitativo, y tomar una decisión explícita. Este paso termina con una decisión escrita, no con una sensación.

3.7 Métricas: de vanidad y accionables

El paso 5 depende por completo de **qué medís**, y acá se juega buena parte del valor del ejercicio.

Las **métricas de vanidad** son las que siempre suben y nunca informan: cantidad total de usuarios registrados, visitas acumuladas, descargas, seguidores, “impresiones”. Son cómodas porque el gráfico va para arriba y quedan bien en una lámina. El problema es que no permiten tomar ninguna decisión: si los registros crecen pero nadie usa el producto dos veces, el número es un espejismo.

Las **métricas accionables** cumplen tres condiciones: son **accionables** (un cambio en el producto las mueve de forma comprensible), **accesibles** (todo el equipo entiende cómo se calculan) y **auditables** (se puede volver al dato original y verificarlo). Suelen ser tasas, no totales: conversión, retención a 7 días, frecuencia de uso, porcentaje de usuarios que completan el flujo principal.

Un ordenador útil es el embudo **AARRR** (McClure, 2007), conocido como “métricas pirata”:

Etapas	Pregunta	Ejemplo de métrica
Adquisición	¿Cómo te encuentran?	Visitantes únicos por canal, costo por visita
Activación	¿Tienen una primera experiencia satisfactoria?	% que completa el registro y usa la función principal
Retención	¿Vuelven?	% que vuelve a los 7 y 30 días
Ingresos (Revenue)	¿Pagan?	% que convierte a pago, ticket promedio
Referencia	¿Lo recomiendan?	% que invita a otro, NPS

Para un MVP en etapa temprana, **la etapa que más importa es la retención**. Adquirir usuarios se puede comprar; que vuelvan, no. Si los usuarios no vuelven, todo lo demás es una fuga que ningún presupuesto de marketing va a tapar.

Una precisión metodológica que separa un análisis serio de uno ingenuo: medí por **cohortes**, no por totales. Una cohorte es el grupo de usuarios que ingresó en un mismo período. Si comparás el comportamiento de los que entraron en marzo con el de los que entraron en abril, podés ver si tus cambios mejoraron algo. Si mirás el total acumulado, el volumen de usuarios nuevos tapa el comportamiento de los viejos y todo parece estar mejorando siempre.

3.8 Los beneficios del MVP, y por qué importan al emprender

Los cuatro beneficios canónicos —que trabajamos en clase— merecen que se les ponga número y consecuencia:

Validación temprana del mercado. Permite obtener retroalimentación de usuarios reales *antes* de invertir recursos adicionales. El valor concreto es de opción: cada semana que no gastás en construir lo que no se va a usar es capital disponible para la apuesta siguiente.

Ahorro de tiempo y recursos. Evita desarrollar funcionalidades innecesarias. Distintos estudios de la industria del software coinciden en que una fracción mayoritaria de las funcionalidades desarrolladas se usa raramente o nunca. Cada una de ellas costó análisis, desarrollo, testing y —lo más caro— mantenimiento perpetuo.

Reducción de riesgos. Minimiza la probabilidad de lanzar algo que no encuentre tracción. Volviendo al dato de la introducción: el 35 % de las startups muere por ausencia de necesidad de mercado (CB Insights, 2021). El MVP es, literalmente, el mecanismo diseñado para detectar eso antes de gastar el capital.

Aprendizaje rápido. Genera insights que mejoran el producto en versiones futuras. Y hay un efecto de segundo orden: acelera el ritmo de aprendizaje del **equipo**, que es el activo que efectivamente se acumula.

A esos cuatro conviene sumarle dos que rara vez se mencionan y que a vos, en el contexto del Taller, te van a resultar muy concretos:

Credibilidad ante inversores. Un pitch que dice “creemos que hay demanda” y un pitch que dice “lanzamos a 40 usuarios en dos barrios, el 62 % volvió a comprar en la segunda semana y el ticket promedio fue de X” no compiten en la misma categoría. El segundo transformó una opinión en evidencia. En la rúbrica del Pitch Final, esto se juega directamente en los criterios de propuesta de valor y de viabilidad.

Foco del equipo. Un MVP obliga a decidir qué queda afuera. Esa conversación —incómoda, a veces conflictiva— es la que convierte un grupo de cinco personas con cinco ideas distintas en un equipo con una sola dirección.

3.9 La decisión: iterar, pivotar o perseverar

Cada vuelta del ciclo termina en una decisión de tres opciones:

- **Perseverar:** los datos confirman la hipótesis. Se profundiza en la dirección actual.
- **Iterar:** los datos son parcialmente favorables. Se ajusta el producto y se vuelve a medir.
- **Pivotar:** los datos refutan una hipótesis estructural. Se cambia de dirección **conservando lo aprendido**.

Un pivote no es empezar de cero ni admitir un fracaso: es un cambio estructurado que mantiene un pie en lo validado. Ries identifica varios tipos, entre los que vale la pena conocer: el **pivote de segmento** (mismo producto, cliente distinto), el **pivote de problema** (mismo cliente, problema distinto), el **pivote de acercamiento** (una funcionalidad secundaria resulta ser el producto), el **pivote de modelo de ingresos** y el **pivote de canal**.

¿Cuándo pivotar? La respuesta honesta es: **más temprano de lo que te va a gustar**. Los equipos tienden a persistir demasiado, por dos razones bien documentadas: la falacia del costo hundido —“ya invertimos ocho meses”— y el enamoramiento de la solución. Un antídoto práctico es agendar una **reunión de pivotar-o-perseverar** cada cierto número de semanas, fija, donde se revisan los umbrales que se habían escrito *antes* de los experimentos. Ponerle fecha al análisis evita que la decisión dependa del ánimo del equipo ese día.

Sobre el costo emocional conviene ser directo: matar tu propia idea es difícil, y en un contexto académico donde además hay una nota en juego, más todavía. Pero el criterio profesional es el que enuncia el caso de PedidosYa Créditos que se presenta en clase: **enamorate del problema, no de la solución**. Si el problema es real, hay muchas soluciones posibles y vos tenés ahora información que nadie más tiene.

3.10 Tres casos que vale la pena conocer

Dropbox: el MVP que era un video. En 2007, Drew Houston enfrentaba un problema antipático: construir un sistema de sincronización confiable entre múltiples sistemas operativos era largo y caro, y la propuesta de valor costaba de explicar en palabras —la gente ya tenía pendrives y no entendía por qué necesitaría otra cosa—.

La solución fue un **video de demostración de unos tres minutos**, dirigido a la comunidad técnica temprana, que mostraba el producto funcionando como si ya existiera: una carpeta en el escritorio; todo lo que se pone ahí se sincroniza solo en las demás computadoras; y se pueden compartir enlaces a esos archivos. El video circuló y, de la noche a la mañana, **la lista de espera pasó de unos 5.000 a unos 75.000 interesados** (Ries, 2011).

Lo que se validó no fue la tecnología —Houston sabía que era factible— sino **la demanda**. El video no era marketing: era un experimento diseñado para responder una pregunta concreta, y su resultado habilitó la decisión de invertir años de desarrollo.

Airbnb: el concierge más artesanal del mundo. En 2007, dos diseñadores que no podían pagar el alquiler en San Francisco inflaron tres colchones en su living y ofrecieron alojamiento a asistentes de una conferencia de diseño que había dejado los hoteles sin lugar. Sin plataforma, sin sistema de pagos, sin algoritmo: un sitio armado a mano y tres huéspedes. Más adelante, con la plataforma ya existente pero sin tracción en Nueva York, los fundadores hicieron algo que ninguna startup “escalable” haría: **viajaron, golpearon las puertas de sus anfitriones y fotografiaron los departamentos ellos mismos**. Los ingresos de la ciudad se multiplicaron. Paul Graham resumió el principio: al principio hay que **hacer cosas que no escalan** (Graham, 2013), porque el objetivo de esa etapa no es la eficiencia operativa sino entender de primera mano qué necesita el usuario.

El patinete de Kniberg: cómo se corta un producto. Henrik Kniberg (2016), coach ágil que trabajó con Spotify, popularizó una imagen que se volvió el mejor resumen visual del concepto. En la fila de arriba, el desarrollo por partes: primero una rueda, después dos, después el chasis, y recién al final un auto. El usuario no tiene nada utilizable hasta el último paso, y no hay feedback posible en el camino. En la fila de abajo, el mismo destino final, pero cada etapa entrega algo que **transporta a alguien**: un patinete, después una bicicleta, después una moto, finalmente el auto.

La diferencia no es la velocidad: es que en la fila de abajo el usuario puede usar algo desde el primer día, y el equipo aprende en cada etapa. Quizá descubra que sus usuarios recorren dos kilómetros por ciudad y que la bicicleta ya resolvía el problema — con lo cual el auto nunca hizo falta.

El punto fino, y la razón por la que Kniberg prefiere hablar de *earliest testable / usable / lovable product*, es que el patinete y la rueda **no son comparables en tamaño**: son comparables en que uno entrega valor y el otro no. Cortar un producto no es cortarlo en pedazos: es cortarlo en versiones completas y chiquitas de sí mismo.

Conexión con la bibliografía

Esta sección debe leerse en tándem con los capítulos 3, 6 y 7 de **Ries (2011)**: aprendizaje validado, testear y medir, respectivamente. Como se profundiza en Ries, esto significa que la pregunta “¿deberíamos construir esto?” precede siempre a “¿podemos construir esto?”, y que la mayoría de los emprendedores invierte todo su esfuerzo en la segunda.

Complementariamente, el artículo de **Blank (2013)** —*Why the Lean Start-Up Changes Everything*— ubica al MVP dentro del proceso más amplio de *customer development* y aporta una de las frases fundacionales del campo: ningún plan de negocios sobrevive al primer contacto con los clientes. Si te interesa profundizar, el marco completo está desarrollado en **Blank y Dorf (2012)**, *El manual del emprendedor*, que figura en la bibliografía optativa del programa.

Y una conexión hacia atrás: el MVP presupone el trabajo de la **Unidad 1**. Sin mapa de empatía y sin propuesta de valor canvas no tenés hipótesis que testear — tenés una corazonada. Releé tu propio canvas y preguntate cuál de todas las afirmaciones que escribiste ahí es la más riesgosa. Esa es la que tu MVP tiene que atacar primero.

Mito vs. realidad: el caso del intrapreneur

Se suele asociar el MVP con startups en un garage. El caso de **PedidosYa Créditos**, presentado en clase por Gonzalo Ruesta (2024), muestra que la lógica es idéntica —y a veces más exigente— dentro de una empresa grande.

El equipo detectó una oportunidad: ofrecer financiamiento a los comercios de la plataforma, aprovechando una ventaja que ningún banco tenía —información transaccional propia para el scoring crediticio y la posibilidad de cobrar las cuotas del flujo de ventas semanal del propio comercio. En lugar de construir el sistema completo, definieron un alcance mínimo y **dejaron deliberadamente afuera** la automatización de comunicaciones (se reemplazó por mails manuales y un banner), la cobranza propia (se tercerizó) y la gestión contable robusta (se resolvió con bots de RPA). Lanzaron con **friends & family: unos 1.000 comercios**, para validar los flujos operativos con riesgo controlado.

El aprendizaje llegó rápido y fue incómodo: el flujo de cobranza generaba fricción, los montos eran demasiado bajos, los comercios no entendían los vencimientos y el awareness de la oferta era pobre. Cada dolor se convirtió en un accionable —cobranza automatizada desde la liquidación de ventas, cuotas semanales en lugar de mensuales, más información en el portal—. Más adelante exploraron un modelo alternativo con una entidad financiera en otro país, **fracasaron y tomaron el aprendizaje**. Recién cuando la operación manual se volvió el límite del crecimiento, invirtieron en arquitectura de microservicios, integraciones por API y sistemas de control.

Tres cosas para retener. Primero: el orden fue **entender → lanzar → aprender e iterar → explorar → escalar**, y no al revés. Segundo: el intrapreneur tiene una dificultad adicional que el emprendedor independiente no tiene —debe convencer internamente a los líderes de la organización de que su proyecto es viable y complementario, no una amenaza—, y un MVP con datos reales es el argumento más fuerte disponible. Tercero: **escalar antes de tiempo es tan riesgoso como no escalar nunca**; identificar el momento en que la solución técnica se vuelve un limitante para crecer es una decisión de negocio, no una decisión de ingeniería.

3.11 Límites y críticas del MVP

Sería deshonesto presentar el MVP como una herramienta universal. Tiene límites reales:

Hay industrias donde no aplica tal cual. En dispositivos médicos, aviación, infraestructura crítica o productos farmacéuticos, lanzar una versión incompleta a usuarios reales no es una estrategia sino un delito. Ahí el aprendizaje temprano se busca con simulaciones, ensayos controlados y prototipos de laboratorio. Lo mismo vale, por razones distintas, en fintech regulada: podés testear el flujo, no podés testear el cumplimiento normativo.

El riesgo reputacional es real. La frase de Reid Hoffman —“*si no te avergüenza la primera versión de tu producto, lo lanzaste muy tarde*”— es una corrección saludable al perfeccionismo, y una pésima licencia si se toma literalmente. En mercados con alta competencia y bajo costo de cambio, una primera experiencia mala puede costarte al usuario para siempre. Por eso se habla también de **MLP** (*Minimum Lovable Product*): el piso mínimo no es solo que funcione, sino que quien lo use quiera volver.

Hay productos cuyo valor está en la escala. Un marketplace o una red social sin masa crítica no entrega su propuesta de valor: el MVP de un marketplace con diez usuarios de cada lado no prueba casi nada sobre el marketplace con diez mil. La solución habitual es acotar geográficamente —un barrio, una facultad, una ciudad— para alcanzar densidad en un territorio chico en lugar de dispersión en uno grande.

Y hay un riesgo de método: el MVP mide bien lo que se puede medir rápido y mide mal lo que se manifiesta en el largo plazo. Un experimento de dos semanas no te va a decir nada sobre la retención a dos años, ni sobre si tu marca construye lealtad. Conviene saber qué preguntas tu experimento **no** responde.

Puente a la Unidad 5: del MVP al VAN

Este recuadro es, probablemente, el más importante de la Nota.

En la Unidad 5 vas a construir un flujo de fondos y calcular un VAN, y en el Pitch Final se te va a exigir un VAN positivo a una tasa del 30 % y superior a US\$ 600.000. Ese número va a salir de una planilla, y esa planilla se apoya en supuestos. **Cada supuesto es una hipótesis que tu MVP puede validar o refutar.**

Supuesto del flujo de fondos	Hipótesis que lo sostiene	Qué mide el MVP
Cantidad de clientes año 1	Tasa de penetración sobre el mercado alcanzable	% de usuarios del piloto que se registran
Ingreso por cliente	Ticket promedio y frecuencia de compra	Ticket y compras por usuario por mes
Crecimiento interanual	Mecanismo de crecimiento y tasa de referencia	% de usuarios que invitan a otro
Costo de adquisición (CAC)	Canal y costo por conversión	Gasto del piloto dividido usuarios adquiridos

Supuesto del flujo de fondos	Hipótesis que lo sostiene	Qué mide el MVP
Vida del cliente (para el LTV)	Retención mes a mes	% que sigue activo a 30, 60 y 90 días
Costos operativos variables	Costo real de servir a un cliente	Costo del concierge dividido usuarios atendidos

La diferencia entre un proyecto que aprueba con comodidad y uno que se cae en la ronda de preguntas suele estar exactamente acá. Cuando el jurado pregunte “¿de dónde sacaste el 25 % de penetración?”, hay dos respuestas posibles. Una es “lo estimamos”. La otra es “lo medimos en un piloto de ocho semanas en dos barrios, con este resultado”. Solo la segunda sobrevive a la repregunta.

? Preguntas disparadoras

1. Escribí, con la plantilla de la sección 3.4, la hipótesis **más riesgosa** de tu proyecto. ¿Qué pasa con todo el modelo si resulta falsa?
2. De los seis tipos de MVP de la tabla, ¿cuál despeja ese riesgo con menor esfuerzo? ¿Por qué descartás los otros cinco?
3. Si tu MVP funciona perfectamente y nadie vuelve a usarlo en la segunda semana, ¿qué aprendiste? ¿Es un resultado peor que si nadie lo hubiera usado nunca?
4. ¿Cuál es la métrica de vanidad que tu equipo estaría más tentado de mostrar en el pitch? ¿Con qué métrica accionable la reemplazarías?
5. Definí ahora, por escrito, el umbral que los llevaría a pivotar. Si no pueden acordarlo, ¿qué les dice eso sobre la claridad de su hipótesis?

Parte IV · Design Thinking

4.1 Qué es, y de dónde viene

El **Design Thinking** es un enfoque de resolución de problemas centrado en las personas, que aplica la sensibilidad y los métodos del diseñador para satisfacer necesidades humanas con lo que es tecnológicamente factible y comercialmente viable. La definición es, esencialmente, la de Tim Brown (2008), y conviene leerla despacio porque contiene las tres restricciones que lo definen todo:

- **Deseabilidad:** ¿alguien lo quiere?
- **Factibilidad:** ¿se puede construir?
- **Viabilidad:** ¿es sostenible como negocio?

Una innovación existe solo en la intersección de las tres. Un producto deseable y factible pero inviable es un hobby caro. Uno factible y viable pero indeseable es el 35 % de las startups que mueren por falta de mercado. Y uno deseable y viable pero infactible es una presentación de PowerPoint.

El nombre es reciente pero la idea no. Herbert Simon (1969) ya distinguía entre las ciencias que estudian lo que *existe* y las “ciencias de lo artificial”, que se ocupan de **transformar situaciones existentes en situaciones preferidas** — que es, palabra por palabra, la definición

de diseñar. La formalización como método de gestión llegó con la consultora **IDEO** en los años noventa y con la **d.school de Stanford**, que sistematizó las cinco etapas que hoy se enseñan en todo el mundo.

El desplazamiento conceptual importante es este: el diseño dejó de entenderse como una etapa estética al final del proceso —darle forma bonita a lo que ya se decidió— para convertirse en una **forma de investigar el problema desde el principio**.

4.2 La lógica divergente–convergente y el doble diamante

El Design Thinking alterna dos modos de pensamiento que son opuestos y que la mayoría de los equipos mezcla, con malos resultados:

- **Divergente:** abrir, generar muchas opciones, suspender el juicio, buscar cantidad y variedad.
- **Convergente:** cerrar, evaluar, descartar, decidir.

Diverger y converger al mismo tiempo es la receta para las reuniones improductivas: alguien propone una idea y otro explica por qué no funciona, y en veinte minutos nadie propone nada más. El método exige separarlos explícitamente en el tiempo.

El **doble diamante**, formulado por el Design Council británico (2019), organiza esa alternancia en cuatro fases:

Fase	Modo	Pregunta	Salida
Descubrir	Divergente	¿Qué está pasando realmente?	Investigación, observaciones, datos crudos
Definir	Convergente	¿Cuál es el problema que vale la pena resolver?	Un problema bien planteado
Desarrollar	Divergente	¿Qué podríamos hacer?	Muchas soluciones candidatas
Entregar	Convergente	¿Qué funciona?	Una solución probada

El primer diamante trabaja sobre el **problema**; el segundo, sobre la **solución**. La mayoría de los equipos se saltea el primero entero: empieza en “desarrollar” porque ya tiene la solución en la cabeza desde el día uno. La consecuencia es previsible — se resuelve con enorme prolijidad un problema que nadie tenía.

4.3 Las cinco etapas

1. Empatizar. Comprender a las personas para las que se diseña, en su contexto y en sus propias palabras. Se hace observando y preguntando: entrevistas en profundidad, observación de campo, acompañamiento de un usuario durante su tarea real. La regla de oro es preguntar por **comportamientos pasados y concretos**, no por intenciones futuras. “¿Usarías una app que...?” siempre se responde que sí; “contame la última vez que necesitaste X: ¿qué hiciste?” produce información utilizable. *Salida: comprensión profunda del usuario, en sus términos.*

2. Definir. Sintetizar lo observado en un enunciado claro de problema. El formato clásico es el **POV** (*point of view*): *[usuario específico] necesita [necesidad, expresada como verbo] porque [insight sorprendente descubierto en la investigación]*. La clave está en el tercer elemento: si el “porque” es obvio, no investigaste lo suficiente. *Salida: un problema bien planteado.*

3. Idear. Generar la mayor cantidad posible de soluciones, sin evaluarlas. Las técnicas son variadas —**How Might We** para convertir el POV en preguntas generativas, **Crazy 8s** para forzar ocho ideas en ocho minutos, brainstorming con reglas explícitas— pero todas comparten el mismo principio: **separar la generación del juicio**. La cantidad es un objetivo en sí misma, porque las primeras ideas de un grupo son siempre las convencionales. *Salida: un abanico amplio de opciones.*

4. Prototipar. Construir versiones rápidas, baratas y desechables de las ideas más prometedoras. Un prototipo puede ser un dibujo en papel, una secuencia de pantallas en una herramienta de diseño, un storyboard o una dramatización del servicio. La regla: **el prototipo debe ser lo suficientemente barato como para que no te duela tirarlo**. Si te duele, vas a defenderlo en lugar de testearlo. *Salida: algo tangible con lo que el usuario pueda interactuar.*

5. Testear. Poner el prototipo frente a usuarios reales y observar. No explicarlo, no defenderlo: entregárselo y callarse. Las mejores observaciones aparecen en los silencios, en las dudas y en los errores del usuario. Nielsen (2000) mostró que **cinco usuarios detectan alrededor del 85 % de los problemas de usabilidad**: no necesitas una muestra estadística para encontrar lo que está roto. *Salida: aprendizaje que alimenta la próxima vuelta.*

Las cinco etapas **no son una secuencia lineal**. Se vuelve constantemente: testear devuelve a idear, y a veces a empatizar. Dibujarlas como una flecha recta es una simplificación de las láminas.

4.4 La caja de herramientas, por etapa

Etapa	Herramienta	Para qué sirve
Empatizar	Entrevista en profundidad	Entender motivaciones y contexto; 45–60 minutos, preguntas abiertas
Empatizar	Mapa de empatía	Organizar lo que el usuario <i>piensa, siente, ve, oye, dice y hace</i> , más dolores y ganancias
Definir	POV statement	Enunciar el problema en una frase accionable
Definir	Propuesta de valor canvas	Alinear dolores y alegrías del cliente con aliviadores y creadores del producto
Idear	How Might We	Convertir un problema en una pregunta que invite a soluciones

Etapa	Herramienta	Para qué sirve
Idear	Crazy 8s	Forzar volumen y romper la primera idea obvia
Prototipar	Prototipo de papel / wireframe	Testear un flujo sin escribir código
Testear	Test de usabilidad con 5 usuarios	Detectar la mayoría de los problemas con costo mínimo

Las dos herramientas en negrita del bloque de empatía y definición ya las usaste en la Unidad 1. Vale la pena que veas la continuidad: **el mapa de empatía y la propuesta de valor canvas no eran ejercicios de inicio de cuatrimestre, eran las etapas 1 y 2 de este método.**

4.5 Por qué es tan útil cuando recién empezás

Esta es la pregunta específica del programa, y merece una respuesta en serio. Seis razones:

- 1. Baja el costo del error cuando es más caro equivocarse.** Al principio no tenés capital, no tenés marca y no tenés segundas oportunidades. El Design Thinking mueve el descubrimiento de errores hacia la etapa en que corregir cuesta una hoja de papel en lugar de seis meses de desarrollo.
- 2. Te saca del enamoramiento de la solución.** Casi todo emprendedor arranca con una solución en la cabeza —una app, una plataforma, un dispositivo— y recién después busca un problema que la justifique. El método invierte el orden por diseño: te obliga a pasar por Empatizar y Definir antes de tener derecho a proponer nada.
- 3. Sustituye autoridad por evidencia.** En un equipo sin jerarquías —cinco compañeros de cursada, o tres socios fundadores— las discusiones de producto se resuelven por insistencia o por carisma. Tener un método compartido y datos de usuarios reales cambia la naturaleza de la discusión: se deja de debatir sobre opiniones.
- 4. Hace tangible lo intangible.** Un producto digital que no existe es difícil de discutir. Un prototipo en papel, por tosco que sea, permite que cinco personas hablen sobre la misma cosa. Buena parte de los desacuerdos de equipo desaparecen cuando se descubre que cada uno imaginaba un producto distinto.
- 5. Genera el material del pitch.** Las entrevistas producen citas textuales, historias y datos. Un pitch que abre con la frase literal de un usuario describiendo su problema es incomparablemente más potente que uno que abre con una estadística de mercado. En la rúbrica del Pitch Final, el criterio de *problema, cliente y propuesta de valor* se alimenta directamente de este trabajo.
- 6. Te entrena en tolerancia a la ambigüedad.** Es una habilidad, y se aprende. El método te fuerza a sostener un problema abierto sin cerrarlo prematuramente — que es exactamente lo que la incertidumbre de emprender va a exigirte todo el tiempo.

4.6 El continuo: Design Thinking, Lean Startup y Agile

Los tres marcos de esta unidad suelen presentarse como alternativas entre las que hay que elegir. No lo son: son **secuenciales y complementarios**, y cada uno responde una de las tres restricciones de Brown.

	Design Thinking	Lean Startup / MVP	Agile (Scrum / Kanban)
Pregunta central	¿Estamos resolviendo el problema correcto?	¿Alguien lo quiere y lo pagaría?	¿Cómo lo construimos bien y a tiempo?
Restricción que ataca	Deseabilidad	Viabilidad	Factibilidad
Unidad de trabajo	El insight	La hipótesis	El incremento
Modo de prueba	Prototipo frente a usuarios	Experimento en el mercado	Software funcionando en producción
Momento	Discovery y Definition	Definition y Development	Development y Delivery
Producto	Un problema bien planteado	Un modelo de negocio validado	Un producto construido y mantenible
Riesgo de usarlo solo	Insights que nunca se ponen a prueba	Experimentos rigurosos sobre el problema equivocado	Entregar eficientemente lo que nadie usa

Leído de izquierda a derecha, el cuadro describe el recorrido completo de un producto digital — y coincide, no por casualidad, con las cuatro fases de Discovery, Definition, Development y Delivery que planteamos en la introducción.

4.7 Críticas honestas

El Design Thinking es probablemente el marco de esta unidad que más ha sufrido de éxito, y las críticas merecen escucharse.

Bruce Nussbaum (2011), uno de sus principales difusores, publicó un texto con un título provocador: *“Design Thinking is a failed experiment. So what’s next?”*. Su argumento no era que el método estuviera mal, sino que al empaquetarse en un proceso de cinco pasos para consultoras se le extrajo lo esencial —la incomodidad, la exploración genuina, el riesgo— y quedó un ritual que produce la sensación de innovar sin sus costos. Las tres objeciones más sólidas:

- **El teatro de la innovación.** Un taller de dos días con post-its no es investigación de usuarios. Si la organización no está dispuesta a cambiar nada de lo que ya decidió, el taller cumple una función decorativa y consume el presupuesto de innovación real.
- **La superficialidad de la empatía express.** Cinco entrevistas de veinte minutos con conocidos del equipo no constituyen comprensión profunda de un segmento. El riesgo es peor que no investigar: es creer que se investigó.
- **La ausencia de restricciones de negocio.** Idear sin considerar costos, regulación o capacidades reales produce conceptos preciosos e irrealizables. Por eso las tres restricciones de Brown son tres y no una.

La conclusión práctica no es descartar el método sino usarlo con honestidad: **el valor está en el contacto real con usuarios reales, no en el formato del taller.** Si de todo el Design Thinking solo pudieras conservar una cosa, conservá las entrevistas — y hacelas bien.

Conexión con la bibliografía

Esta sección debe leerse en tándem con el artículo fundacional de **Brown (2008)**, *Design Thinking*, y con el **Bootcamp Bootleg** de la d.school (2018), un manual de campo gratuito de unas cincuenta páginas con una ficha por herramienta. Como se profundiza en la lectura de Brown, esto significa que el diseñador no resuelve el problema que le trajeron: **primero lo redefine**, y esa redefinición suele ser el verdadero aporte. Prestá atención a la noción de *pensamiento abductivo* —razonar desde lo que podría ser, no desde lo que es— porque es lo que distingue al diseño de la mera optimización.

Para el proceso completo, el **Framework for Innovation** del Design Council (2019) documenta el doble diamante en su versión actualizada, con acceso libre. Y si vas a hacer entrevistas en serio, el **Field Guide to Human-Centered Design** de IDEO.org (2015) tiene las guías operativas.

Contrapunto obligatorio: leé también a **Nussbaum (2011)**. Una guía de estudio que te presenta un método sin sus críticas no te está enseñando el método: te está vendiendo una herramienta.

Llévalo a la práctica: tu ciclo de descubrimiento en 90 minutos

Antes de escribir una sola línea de código, con tu equipo:

1. **(30 min) Cinco entrevistas.** Uno por integrante, con personas del segmento real —no compañeros de cursada, salvo que sean efectivamente tu segmento—. Una sola consigna: *contame la última vez que tuviste este problema*. Prohibido mencionar su solución.
2. **(20 min) Volcado.** Cada uno escribe en post-its las citas textuales y los comportamientos observados. Citas, no interpretaciones.
3. **(15 min) Agrupamiento.** Junten los post-its por afinidad y pónganle nombre a cada grupo. Los nombres son los patrones.
4. **(10 min) POV.** Escriban **un solo** enunciado: *[usuario] necesita [verbo] porque [insight]*. Si el “porque” no los sorprende a ustedes mismos, vuelvan al paso 3.
5. **(15 min) How Might We + Crazy 8s.** Conviertan el POV en tres preguntas “¿cómo podríamos...?” y generen ocho ideas en ocho minutos sobre la mejor de ellas.

Al final tienen un problema bien planteado y un abanico de soluciones — que es exactamente el insumo que la Parte III necesita para formular la hipótesis del MVP.

Preguntas disparadoras

1. ¿Cuál fue el insight más sorprendente de sus entrevistas? Si no hubo ninguno, ¿qué dice eso sobre cómo preguntaron?

2. ¿Por qué preguntar “¿usarías esto?” produce información inservible? ¿Qué pregunta la reemplaza?
3. Revisen su POV: ¿podría escribirlo igual un competidor que nunca habló con un usuario? Si sí, no es un POV.
4. ¿En cuál de las tres restricciones de Brown está el mayor riesgo de su proyecto hoy: deseabilidad, viabilidad o factibilidad?
5. ¿Cómo distinguirían, en su propio equipo, un proceso de descubrimiento genuino de un “teatro de la innovación”?

Parte V · Integración: del post-it al backlog

5.1 El mapa único

Los cuatro temas de esta unidad se superponen en una sola línea de tiempo. La siguiente tabla los alinea; conviene tenerla a mano cuando armes el roadmap de tu proyecto.

Fase del producto	Marco dominante	Actividad principal	Artefacto que se produce
Discovery	Design Thinking (Descubrir / Empatizar)	Entrevistas, observación, análisis de datos	Mapa de empatía, citas de usuarios
Definition	Design Thinking (Definir) + Lean	Síntesis del problema y de la apuesta	POV, propuesta de valor canvas, hipótesis escrita
Development	Lean + Agile	Definición del alcance mínimo y construcción	Alcance del MVP , backlog, incrementos
Delivery	Lean (Medir)	Puesta en manos de usuarios y medición	Métricas de cohorte, feedback cualitativo
Decisión	Lean (Aprender)	Evaluación contra los umbrales preestablecidos	Iterar, pivotar o perseverar

5.2 Trazabilidad: un insight que se convierte en código

La prueba de que entendiste la unidad es poder recorrer la cadena completa sin saltos. Un ejemplo:

1. **Observación (Empatizar).** En una entrevista, una vecina dice: *“al final termino pidiendo por WhatsApp al almacén y le pago cuando paso, pero nunca sé si tiene lo que necesito”*.
2. **Insight (Definir).** El problema no es la distancia al supermercado: es la **incertidumbre sobre la disponibilidad**. Nadie quiere hacer un viaje en vano.

3. **POV.** *Los residentes de barrios cerrados necesitan confirmar disponibilidad antes de decidir dónde comprar, porque el costo real no es el precio sino el viaje perdido.*
4. **How Might We.** *¿Cómo podríamos mostrar stock en tiempo real sin integrarnos con el sistema del comercio?*
5. **Hipótesis.** *Creemos que mostrar disponibilidad en vivo aumenta la conversión; sabremos que es cierto si el 40 % de quienes consultan stock completan una compra en las 24 horas siguientes.*
6. **Alcance del MVP.** Una lista de 30 productos actualizada manualmente dos veces por día. Nada de integración, nada de tiempo real de verdad. Un MVP tipo **magos de Oz**.
7. **Ítem de backlog.** *Como vecina, quiero ver qué productos hay disponibles hoy para decidir si compro acá.* Criterios de aceptación: la lista carga en menos de 2 segundos, muestra fecha y hora de la última actualización, y funciona en el navegador del celular.
8. **Incremento entregado.** Al cierre del Sprint, funcionando y en manos de usuarios.
9. **Medición.** Conversión de la cohorte de esa semana, comparada con la anterior.
10. **Decisión.** Por encima del 40 %, se persevera y se invierte en la integración real. Por debajo, se vuelve al paso 2: quizá el problema no era la disponibilidad.

Diez pasos, sin ningún salto de fe en el medio. Ese es el estándar.

5.3 Checklist para el Pre-Pitch y el Pitch Final

Marcá cada punto que tu equipo pueda responder hoy, sin improvisar:

☐	Punto a verificar
☐	Podemos citar textualmente a tres usuarios reales describiendo el problema
☐	Tenemos escrito un POV que sorprende a alguien externo al equipo
☐	Tenemos escrita la hipótesis más riesgosa , con métrica, umbral y plazo
☐	Sabemos qué decisión tomaríamos si el umbral no se alcanza
☐	Podemos explicar en una frase qué queda afuera de nuestro MVP y por qué
☐	Elegimos un tipo de MVP y podemos justificar por qué no elegimos los otros
☐	Definimos tres métricas accionables y ninguna de ellas es un total acumulado
☐	Sabemos qué supuesto del flujo de fondos valida cada métrica
☐	Podemos explicar por qué elegimos nuestro enfoque metodológico, con los tres criterios de la sección 1.8

☐	Punto a verificar
☐	Si decimos “Scrum” o “Kanban”, podemos describir nuestros eventos, artefactos y límites de WIP reales
☐	Tenemos una Definition of Done escrita de no más de cinco puntos
☐	Podemos narrar la cadena completa de la sección 5.2 con nuestro propio proyecto

5.4 Los seis errores más frecuentes en los pitches

1. **El “MVP” que es el producto completo.** Cinco funcionalidades, tres perfiles de usuario y un panel de administración. Eso no es un mínimo: es el plan de seis meses con otro nombre.
2. **Hipótesis no falsables.** “Creemos que hay una gran oportunidad de mercado.” No hay forma de que eso resulte falso, y por lo tanto no hay forma de que resulte verdadero.
3. **Métricas de vanidad en la lámina de tracción.** Descargas totales, visitas acumuladas, seguidores. La repregunta siempre es la misma y siempre duele: *¿cuántos volvieron?*
4. **“Usamos metodologías ágiles” como contraseña.** Se enuncia y no se puede sostener. Si el jurado pregunta cuánto dura el sprint y qué pasa si algo no entra, hay que poder responder.
5. **Roadmap sin evidencia.** Un cronograma prolijo a dieciocho meses con funcionalidades decididas de antemano contradice todo lo que se acaba de decir sobre iterar y aprender.
6. **Supuestos financieros sin origen.** Una penetración del 30 % que apareció en la planilla sin que nadie sepa de dónde salió. Es el punto donde se pierden más puntos en el criterio de viabilidad económico-financiera.

5.5 Qué criterios de la rúbrica se juegan acá

Esta unidad impacta directamente en tres de los cinco criterios de evaluación del Pitch Final:

- **Problema, cliente y propuesta de valor (15 %):** se alimenta del trabajo de Design Thinking de la Parte IV.
- **Viabilidad económico-financiera (25 %):** los supuestos de tu flujo de fondos se sostienen —o no— en la evidencia que generó tu MVP.
- **Defensa en la ronda de preguntas (25 %):** es el criterio individual, y es donde se nota si entendiste el método o memorizaste el vocabulario.

Sumados, más de la mitad de la nota del Pitch Final. Vale la pena el esfuerzo.

Síntesis: diez ideas fuerza

1. **La principal causa de muerte de un producto digital no es construirlo mal, sino construir bien lo que nadie pidió.**

2. Cascada y Agile no son buenos o malos en abstracto: se eligen según nivel de incertidumbre, costo del error y velocidad de cambio del entorno.
3. El Manifiesto Ágil no niega los planes, la documentación ni los contratos: establece una prioridad cuando entran en conflicto con el aprendizaje.
4. Scrum es un marco deliberadamente incompleto que se apoya en el empirismo: transparencia, inspección y adaptación.
5. En Kanban, **limitar cuánto se empieza es lo que hace que las cosas se terminen**. La Ley de Little lo explica sin apelar a la fuerza de voluntad.
6. El MVP optimiza **aprendizaje validado por unidad de esfuerzo**, no funcionalidades ni costos.
7. Una hipótesis que no puede resultar falsa no es una hipótesis. El umbral de fracaso se define **antes** del experimento.
8. La retención es la métrica que más importa en etapa temprana: los usuarios se pueden comprar, la recurrencia no.
9. Enamorate del problema, no de la solución. Pivotar no es fracasar: es usar información que antes no tenías.
10. Design Thinking, Lean y Agile responden tres preguntas distintas —deseabilidad, viabilidad, factibilidad— y ninguno de los tres funciona bien solo.

Glosario

AARRR: embudo de métricas (Adquisición, Activación, Retención, Revenue, Referencia). **Agile:** conjunto de valores y principios para el desarrollo adaptativo; no es una metodología. **Backlog:** lista ordenada de todo lo que podría necesitar el producto. **Cascada (Waterfall):** modelo de desarrollo secuencial por fases, con alcance definido al inicio. **Cohorte:** grupo de usuarios que ingresó en un mismo período, analizado en conjunto. **Cycle time:** tiempo entre que el equipo empieza a trabajar en un ítem y lo termina. **Definition of Done (DoD):** estándar de calidad explícito que define cuándo un trabajo está terminado. **Design Thinking:** enfoque de resolución de problemas centrado en las personas, en cinco etapas. **Doble diamante:** modelo de cuatro fases (descubrir, definir, desarrollar, entregar) que alterna divergencia y convergencia. **Historia de usuario:** formato de requisito que expresa quién, qué y para qué. **Hipótesis de valor:** supuesto sobre si el producto entrega valor real a quien lo usa. **INVEST:** criterios de calidad de un ítem de backlog (independiente, negociable, valioso, estimable, pequeño, testeable). **Kanban:** marco de gestión del flujo basado en visualización y límites de trabajo en curso. **Ley de Little:** relación entre trabajo en curso, throughput y tiempo de ciclo. **MLP (Minimum Lovable Product):** versión mínima que además genera adhesión, no solo funcionalidad. **MVP:** versión de un producto nuevo que permite obtener el máximo aprendizaje validado con el mínimo esfuerzo. **Métrica accionable:** métrica que permite tomar decisiones; opuesta a la métrica de vanidad. **Pivote:** cambio estructurado de dirección que conserva lo aprendido. **POV (Point of View):** enunciado sintético del problema: usuario + necesidad + insight. **Product Owner:** responsable de maximizar el valor del producto y del orden del backlog. **Retención:** proporción de usuarios que vuelven a usar el producto tras un período. **Scrum:** marco de trabajo ágil basado en Sprints, con tres responsabilidades, cinco eventos y tres artefactos. **Sprint:** período de duración fija, de un mes o menos, que contiene

todo el trabajo de Scrum. **Sprint Goal:** objetivo único que da coherencia a un Sprint. **WIP (Work in Progress):** trabajo empezado y no terminado.

Anexo A · Plantillas de trabajo

A.1 Plantilla de hipótesis

Creemos que (segmento específico) tiene el problema de (problema concreto) y que al ofrecerle (solución acotada) va a (comportamiento observable) Sabremos que estábamos en lo cierto si (métrica) alcanza (umbral) en (plazo). Si no lo alcanza, (decisión).

A.2 Priorización del alcance del MVP

Para cada funcionalidad candidata, completá la fila y ordená por la última columna:

Funcionalidad	¿Qué hipótesis valida?	Esfuerzo (1-5)	Riesgo que despeja (1-5)	Prioridad = Riesgo / Esfuerzo

Todo lo que no valide ninguna hipótesis queda afuera del MVP. Sin excepciones.

A.3 Tablero Kanban inicial

Columna	Límite de WIP	Política de entrada	Política de salida
Por hacer	—	Priorizado por el PO	—
En curso	(nº de integrantes disponibles)	Hay capacidad libre	Cumple la DoD parcial
En revisión	2	Revisado por otro integrante	Aprobado
Terminado	—	Cumple la DoD completa	—

Referencias

- Anderson, D. J., & Carmichael, A. (2016). *Essential Kanban condensed*. Lean Kanban University Press. <https://leankanban.com/guide/>
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifiesto por el desarrollo ágil de software*. <https://agilemanifesto.org/iso/es/manifesto.html>
- Blank, S. (2013). Why the lean start-up changes everything. *Harvard Business Review*, 91(5), 63–72.

- Blank, S., & Dorf, B. (2012). *El manual del emprendedor: La guía paso a paso para crear una gran empresa*. Gestión 2000.
- Brown, T. (2008). Design thinking. *Harvard Business Review*, 86(6), 84–92.
- CB Insights. (2021). *The top 12 reasons startups fail*.
<https://www.cbinsights.com/research/report/startup-failure-reasons-top/>
- Design Council. (2019). *Framework for innovation: Design Council's evolved Double Diamond*. <https://www.designcouncil.org.uk/>
- Graham, P. (2013). *Do things that don't scale*. <https://paulgraham.com/ds.html>
- d.school. (2018). *Bootcamp bootleg*. Hasso Plattner Institute of Design at Stanford University.
<https://dschool.stanford.edu/resources>
- IDEO.org. (2015). *The field guide to human-centered design*. <https://www.designkit.org/>
- Kawasaki, G. (2004). *El arte de empezar*. Ilustreae.
- Kniberg, H. (2016, enero 25). *Making sense of MVP (Minimum Viable Product) – and why I prefer Earliest Testable/Usable/Lovable*. Crisp's Blog.
<https://blog.crisp.se/2016/01/25/henrikkniberg/making-sense-of-mvp>
- McClure, D. (2007). *Startup metrics for pirates: AARRR!* <https://500hats.typepad.com/>
- Nielsen, J. (2000, marzo 19). *Why you only need to test with 5 users*. Nielsen Norman Group.
<https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/>
- Nussbaum, B. (2011, abril 5). Design thinking is a failed experiment. So what's next? *Fast Company*. <https://www.fastcompany.com/1663558/>
- Ries, E. (2011). *El método Lean Startup: Cómo crear empresas de éxito utilizando la innovación continua*. Deusto.
- Royce, W. W. (1970). Managing the development of large software systems. *Proceedings of IEEE WESCON*, 26, 1–9.
- Ruesta, G. (2024). *PedidosYa Créditos: Emprender dentro de una organización* [Presentación de clase]. Taller de Emprendedurismo, Universidad de la Ciudad de Buenos Aires.
- Schwaber, K., & Sutherland, J. (2020). *La guía de Scrum: La guía definitiva de Scrum — Las reglas del juego*. <https://scrumguides.org/>
- Simon, H. A. (1969). *The sciences of the artificial*. MIT Press.
- Thiel, P. (2015). *De cero a uno: Cómo inventar el futuro*. Gestión 2000.
-

Nota sobre el acceso a las fuentes: toda la bibliografía obligatoria de esta unidad —Manifiesto Ágil, Guía Scrum, Essential Kanban Condensed, Brown, Graham, Kniberg, Nielsen, Nussbaum, Design Council, d.school, IDEO.org y CB Insights— es de acceso abierto y gratuito en las direcciones indicadas. Las obras de Ries, Blank y Dorf, Thiel y Kawasaki integran la bibliografía optativa del programa analítico del Taller.